

UJIAN TENGAH SEMSTER
PEMROGRAMAN BERORIENTASI OBJEK



OLEH:

DIMAS ELANG SATRIA NIM : 2409106027

DOSEN : DR. AKHMAD IRSYAD, S.T., M.KOM

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK UNIVERSITAS MULAWARMAN
SAMARINDA

2025

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri game online di Indonesia terus meningkat pesat, salah satunya ditandai dengan popularitas game Wuthering Waves (WUWA). Seiring meningkatnya pemain, kebutuhan akan jasa joki game pun turut berkembang. Jasa joki adalah layanan di mana seorang pemain berpengalaman mengerjakan akun milik klien untuk mencapai target tertentu, seperti membangun karakter, menyelesaikan event, atau mengeksplorasi peta.

Dalam rangka memenuhi Ujian Tengah Semester (UTS) mata kuliah Pemrograman Berorientasi Objek, penulis membuat aplikasi bernama System Management Joki WUWA. Aplikasi ini dirancang untuk membantu penyedia jasa joki mengelola pesanan dari para klien secara terstruktur dan efisien melalui antarmuka berbasis terminal menggunakan bahasa pemrograman Java.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem manajemen pesanan joki game berbasis OOP menggunakan Java?
2. Bagaimana mengimplementasikan perulangan do-while dan for dalam alur program?
3. Bagaimana menerapkan percabangan if-else dan switch-case untuk logika program?
4. Bagaimana mengimplementasikan enkapsulasi, inheritance, dan polimorfisme dalam sistem ini?

1.3 Tujuan

5. Membangun aplikasi manajemen pesanan joki game yang fungsional berbasis Java.
6. Mengimplementasikan seluruh konsep dasar OOP yang dipersyaratkan dalam tugas UTS.
7. Mendemonstrasikan penggunaan perulangan, percabangan, enkapsulasi, inheritance, dan polimorfisme secara nyata dalam sebuah program yang berguna.

1.4 Ruang Lingkup

Program berjalan secara lokal berbasis terminal (command-line). Data pesanan disimpan sementara di memori (ArrayList) selama program berjalan. Terdapat 3 jenis layanan: Build Character, Joki Event, dan Eksplor Map. Program mendukung operasi CRUD: tambah, lihat, update status, dan hapus pesanan.

BAB II LANDASAN TEORI

2.1 Pemrograman Berorientasi Objek (OOP)

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang mengorganisasi perangkat lunak ke dalam objek-objek yang menggabungkan data (atribut) dan perilaku (method). OOP memungkinkan penulisan kode yang lebih modular, mudah dipelihara, dan dapat digunakan kembali (reusable). Empat pilar utama OOP adalah enkapsulasi, inheritance, polimorfisme, dan abstraksi.

2.2 Perulangan (Looping)

Perulangan adalah struktur kontrol untuk mengeksekusi blok kode secara berulang. Java mendukung beberapa jenis perulangan:

- Do-While: Mengeksekusi blok kode minimal satu kali, kemudian memeriksa kondisi. Cocok untuk menu interaktif.
- For: Digunakan ketika jumlah iterasi dapat ditentukan. Mendukung bentuk indexed (for biasa) dan enhanced (for-each).

2.3 Percabangan (Branching)

Percabangan mengalihkan alur eksekusi program berdasarkan kondisi tertentu. Java menyediakan dua struktur utama:

- If-Else: Cocok untuk kondisi dengan rentang nilai atau ekspresi boolean yang kompleks.
- Switch-Case: Cocok untuk kondisi dengan nilai tunggal yang diskrit. Lebih bersih dan mudah dibaca untuk kasus pilihan menu.

2.4 Enkapsulasi (Encapsulation)

Enkapsulasi adalah mekanisme pembungkusan data (atribut) dan method dalam satu unit class, dengan menyembunyikan detail implementasi dari luar. Di Java diterapkan dengan access modifier private pada atribut, serta menyediakan method getter dan setter sebagai antarmuka akses yang terkontrol. Enkapsulasi memungkinkan validasi data dilakukan di dalam setter sebelum nilai disimpan.

2.5 Inheritance (Pewarisan)

Inheritance adalah mekanisme di mana sebuah class (child/subclass) mewarisi atribut dan method dari class lain (parent/superclass) menggunakan keyword extends. Inheritance mendukung prinsip code reuse — kode yang sama tidak perlu ditulis ulang di setiap class turunan. Class turunan dapat menambahkan atribut dan method baru, serta mengoverride method dari class induk.

2.6 Polimorfisme (Polymorphism)

Polimorfisme memungkinkan satu entitas (variabel, method, atau objek) untuk memiliki banyak bentuk. Dalam Java, polimorfisme runtime (dynamic polymorphism) terjadi ketika method yang dioverride di class turunan dipanggil melalui referensi bertipe class induk — Java menentukan method mana yang dipanggil berdasarkan tipe objek sebenarnya pada saat runtime. Constructor overloading adalah bentuk compile-time polymorphism di mana satu class memiliki beberapa constructor dengan parameter berbeda.

BAB III ANALISIS DAN PERANCANGAN

3.1 Gambaran Umum Sistem

System Management Joki WUWA adalah aplikasi CRUD berbasis terminal yang dirancang untuk membantu pengelolaan pesanan jasa joki game Wuthering Waves. Sistem menerima input data akun klien beserta pilihan layanan, menyimpannya dalam ArrayList, dan menyediakan fitur untuk melihat, memperbarui status, dan menghapus pesanan.

3.2 Struktur Class dan Relasinya

Program terdiri dari 7 class dengan hierarki dan relasi sebagai berikut:

Class	Tipe	Keterangan
Layanan	Parent Class	Superclass abstrak yang mendefinisikan atribut bersama (namaGrup, namaPesanan, dataHarga, catatanTambahan) dan method tampilDetail()
BuildCharacter	Child Class	Mewarisi Layanan, menangani layanan build karakter. Memiliki 2 constructor (dengan/tanpa promo)
JokiEvent	Child Class	Mewarisi Layanan, menangani layanan joki event berdasarkan tingkat kesulitan
EksplorMap	Child Class	Mewarisi Layanan, menangani layanan eksplorasi map berdasarkan cakupan area
Harga	Helper Class	Mengelola data harga & kategori. Memiliki 2 constructor (overloading): berdasarkan tipe layanan, dan harga kustom + nama promo
DataJoki	Data Class	Menyimpan data akun klien (username, password, nomorHp) beserta objek Layanan dan status pesanan
Main	Main Class	Entry point program. Berisi menu CRUD utama menggunakan do-while dan switch-case

BAB IV IMPLEMENTASI PROGRAM

4.1 Perulangan — Do-While dan For

Program menggunakan dua jenis perulangan yang berbeda sesuai kebutuhannya masing-masing.

a. Perulangan Do-While (Main.java)

Do-while digunakan untuk menu utama agar program terus berjalan dan menampilkan menu minimal satu kali, sampai user memilih menu 5 (Keluar). Di dalamnya juga terdapat while(true) dengan try-catch untuk memastikan input selalu berupa angka.

```
int menuUtama;
do {
    System.out.println("=====");
    System.out.println("        SYSTEM MANAGEMENT JOKI WUWA");
    System.out.println("=====");
    System.out.println("1. Tambah Pesanan Baru");
    System.out.println("2. Lihat Semua Daftar");
    System.out.println("3. Tandai Selesai (Update)");
    System.out.println("4. Hapus Data (Delete)");
    System.out.println("5. Keluar");

    while (true) { // while untuk validasi input angka
        try {
            System.out.print("Pilih Menu: ");
            menuUtama = inputKeyboard.nextInt();
            inputKeyboard.nextLine();
            break;
        } catch (Exception e) {
            System.out.println("Input harus angka!");
            inputKeyboard.nextLine();
        }
    }
    switch (menuUtama) { ... }

} while (menuUtama != 5); // terus berputar sampai user pilih 5
```

b. Perulangan For — Enhanced dan Indexed (Main.java)

Enhanced for-each digunakan untuk menampilkan daftar pesanan berdasarkan status, sedangkan indexed for digunakan saat perlu nomor urut untuk operasi update dan hapus.

```
// Enhanced for-each - tampil daftar pesanan belum selesai
int nomorAntrian = 1;
for (DataJoki d : daftarPesanan) {
    if (d.getStatus() == 1) {
        System.out.print(nomorAntrian + ". ");
        d.getInfoJoki().tampilDetail(); // Dynamic Polymorphism
        nomorAntrian++;
    }
}
```

```
// Indexed for - tampil daftar dengan nomor untuk operasi hapus/update
for (int i = 0; i < daftarPesanan.size(); i++) {
    System.out.println((i+1) + ". "
        + daftarPesanan.get(i).getUsername()
        + " [" + daftarPesanan.get(i).getInfoJoki().getNamaGrup() +
        "]"");
}
```

4.2 Kondisi dan Percabangan — If-Else dan Switch-Case

a. If-Else di class Harga.java

Percabangan if-else bertingkat digunakan untuk menentukan nilai harga dan nama kategori berdasarkan kombinasi tipe layanan dan tingkat yang dipilih user. Terdapat 9 kombinasi nilai yang mungkin dari 3 tipe x 3 tingkat.

```
public Harga(int pilihan, int tipeLayanan) {
    if (tipeLayanan == 1) { // Build Character
        if (pilihan == 1) { this.nilai = 20000; this.kategori =
            "Leveling Saja"; }
        else if (pilihan == 2) { this.nilai = 45000; this.kategori =
            "Build Echo/Artefak"; }
        else { this.nilai = 80000; this.kategori = "Full Build (Siap
            Pakai)"; }
    } else if (tipeLayanan == 2) { // Joki Event
        if (pilihan == 1) { this.nilai = 15000; this.kategori = "Mudah";
        }
        else if (pilihan == 2) { this.nilai = 30000; this.kategori =
            "Menengah"; }
        else { this.nilai = 50000; this.kategori = "Sulit"; }
    } else { // Eksplor Map
        if (pilihan == 1) { this.nilai = 25000; this.kategori = "Area
            Kecil"; }
        else if (pilihan == 2) { this.nilai = 50000; this.kategori =
            "Area Sedang"; }
        else { this.nilai = 100000; this.kategori = "Full Map"; }
    }
}
```

b. Switch-Case di Main.java

Switch-case menangani 4 menu CRUD utama ditambah menu keluar. Setiap case dibungkus dengan try-catch untuk menangani input yang tidak valid.

```
switch (menuUtama) {
    case 1: // CREATE - Tambah pesanan baru
        // input data akun + pilih layanan + buat objek DataJoki
        break;
    case 2: // READ - Tampilkan semua pesanan
        // tampil antrian (status=1) dan riwayat (status=2)
        break;
    case 3: // UPDATE - Tandai selesai
        // setStatus(2) pada pesanan yang dipilih
        break;
    case 4: // DELETE - Hapus data
        // daftarPesanan.remove(index)
        break;
    case 5:
```

```

        System.out.println("Keluar dari program...");
        break;
    default:
        System.out.println("Menu tidak tersedia!");
    }
}

```

4.3 Enkapsulasi

Enkapsulasi diterapkan pada class `DataJoki` dan `Harga`. Seluruh atribut dideklarasikan `private`, dan akses eksternal hanya diizinkan melalui getter dan setter. Yang menarik adalah setter `setStatus()` pada `DataJoki` yang dilengkapi validasi — status hanya dapat diubah ke nilai 1 (proses) atau 2 (selesai), sehingga tidak ada data yang tidak valid masuk ke sistem.

```

// DataJoki.java - Enkapsulasi dengan validasi pada setter
public class DataJoki {
    private String username, password, nomorHp; // semua atribut
    private
        private int status;
        private Layanan infoJoki;

    // Getter
    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public String getNomorHp() { return nomorHp; }
    public Layanan getInfoJoki() { return infoJoki; }
    public int getStatus() { return status; }

    // Setter dengan validasi
    public void setStatus(int status) {
        // hanya boleh 1 (proses) atau 2 (selesai)
        if (status == 1 || status == 2) {
            this.status = status;
        }
        // jika nilai lain, perubahan diabaikan
    }
}

```

4.4 Inheritance (Pewarisan)

Class `Layanan` berperan sebagai superclass yang mendefinisikan atribut dan method bersama. `BuildCharacter`, `JokiEvent`, dan `EksplorMap` semuanya mewarisi `Layanan` menggunakan keyword `extends`, sehingga dapat langsung menggunakan atribut `namaGrup`, `namaPesanan`, `dataHarga`, dan `catatanTambahan` tanpa mendefinisikannya ulang.

```

// Layanan.java - Parent Class
public class Layanan {
    protected String namaGrup;
    protected String namaPesanan;
    protected Harga dataHarga;
    protected String catatanTambahan = "-";

    public void setCatatan(String catatan) { this.catatanTambahan =
catatan; }
    public String getNamaGrup() { return namaGrup; }
    public String getNamaPesanan() { return namaPesanan; }
    public Harga getDataHarga() { return dataHarga; }
}

```



```

        public void tampilDetail() { // akan di-override child
            System.out.println("Layanan: " + namaGrup);
        }
    }

// BuildCharacter.java – Child Class (mewarisi Layanan)
public class BuildCharacter extends Layanan {
    public BuildCharacter(String nama, int tingkat) {
        this.namaGrup = "Build Char"; // atribut dari parent
        this.dataHarga = new Harga(tingkat, 1); // atribut dari parent
        this.namaPesanan = nama; // atribut dari parent
    }
    // ... (EksplorMap dan JokiEvent sama polanya)
}

```

4.5 Polimorfisme

a. Method Overriding — Dynamic Polymorphism

Setiap class turunan mengoverride method `tampilDetail()` dari `Layanan` dengan format output yang berbeda sesuai konteks layanannya masing-masing:

```

// BuildCharacter – menampilkan kategori build dan harga
@Override
public void tampilDetail() {
    System.out.println("[FOCUS BUILD: " + namaPesanan.toUpperCase() +
    "]\n");
    System.out.println("> Kategori/Promo: " + dataHarga.getKategori());
    System.out.println("> Harga: Rp" + dataHarga.getNilai());
}

// JokiEvent – menampilkan tingkat kesulitan dan catatan
@Override
public void tampilDetail() {
    System.out.println("[EVENT : " + namaPesanan + "]\n");
    System.out.println("> Tingkat Kesulitan: " +
    dataHarga.getKategori());
    System.out.println("> Deadline/Catatan : " + catatanTambahan);
}

// EksplorMap – menampilkan cakupan area dan catatan
@Override
public void tampilDetail() {
    System.out.println("[EKSPLOR MAP: " + namaPesanan + "]\n");
    System.out.println("> Cakupan Area: " + dataHarga.getKategori());
    System.out.println("> Catatan : " + catatanTambahan);
}

```

b. Dynamic Polymorphism saat Runtime (Main.java)

Variabel `layananBaru` bertipe `Layanan` (parent) digunakan untuk menampung objek dari ketiga class turunan. Ketika `tampilDetail()` dipanggil, Java secara otomatis menjalankan implementasi yang sesuai dengan tipe objek aslinya — inilah dynamic polymorphism:

```

// Variabel bertipe parent class menampung berbagai tipe child
Layanan layananBaru;

```

```

if (tipe == 1) {
    layananBaru = new BuildCharacter(namaInput, tingkat); // atau dengan
    promo
} else if (tipe == 2) {
    layananBaru = new JokiEvent(namaInput, tingkat);
} else {
    layananBaru = new EksplorMap(namaInput, tingkat);
}

// Saat dipanggil, Java menentukan method yang tepat berdasarkan objek
aslinya
d.getInfoJoki().tampilDetail(); // Dynamic Polymorphism!

```

c. Constructor Overloading (BuildCharacter.java & Harga.java)

BuildCharacter memiliki dua constructor — satu untuk pesanan biasa, satu untuk pesanan dengan kode promo. Harga juga memiliki dua constructor — satu berdasarkan tipe & tingkat layanan, satu untuk harga kustom dengan nama promo.

```

// BuildCharacter - Constructor Overloading
public BuildCharacter(String nama, int tingkat) {
    // constructor standar tanpa promo
    this.dataHarga = new Harga(tingkat, 1);
}

public BuildCharacter(String nama, int tingkat, String namaPromo) {
    // constructor dengan promo DISKONSAHUR - harga dikurangi 10.000
    this.dataHarga = new Harga(tingkat, 1);
    int hargaDiskon = dataHarga.getNilai() - 10000;
    this.dataHarga = new Harga(hargaDiskon, namaPromo); // Harga
    overloaded
}

// Harga - Constructor Overloading
public Harga(int pilihan, int tipeLayanan) { ... } // berdasarkan tipe
& tingkat
public Harga(int hargaKustom, String namaPromo) { ... } // harga kustom
+ promo

```

BAB V HASIL DAN PEMBAHASAN

5.1 Tampilan Output Program

Berikut adalah simulasi output program saat dijalankan:

```
=====
      SYSTEM MANAGEMENT JOKI WUWA
=====
1. Tambah Pesanan Baru
2. Lihat Semua Daftar
3. Tandai Selesai (Update)
4. Hapus Data (Delete)
5. Keluar
Pilih Menu: 1

--- INPUT DATA AKUN ---
Username Akun: player123
Password Akun: pass123
Nomor HP/WA : 08123456789

--- PILIH TIPE JOKI ---
1. Build Character  2. Joki Event  3. Eksplor Map
Pilih: 1
Masukkan Kode Promo: DISKONSAHUR
Nama (Karakter/Event/Map): Rover

--- PILIH TINGKAT LAYANAN ---
1. Leveling Saja (20rb)
2. Build Echo/Artefak (45rb)
3. Full Build (80rb)
Pilih: 3
>> Berhasil Menambahkan Pesanan!

--- [ DAFTAR BELUM SELESAI ] ---
1. [FOCUS BUILD: ROVER]
  > Kategori/Promo: PROMO: Diskon Sahur
  > Harga: Rp70000
  [Akun: player123 | Pass: pass123 | WA: 08123456789]
```

5.2 Pembahasan

Program berhasil mengimplementasikan semua konsep OOP yang dipersyaratkan. Penggunaan do-while untuk menu utama terbukti tepat karena menu perlu ditampilkan minimal satu kali sebelum kondisi dicek. Kombinasi dengan while(true) dan try-catch untuk validasi input membuat program lebih robust terhadap input yang tidak valid.

Enkapsulasi pada DataJoki berperan penting dalam menjaga integritas data — setter setStatus() dengan validasi memastikan bahwa status pesanan hanya bisa bernilai 1 atau 2, mencegah data yang tidak konsisten masuk ke sistem.

Inheritance memungkinkan ketiga class layanan (BuildCharacter, JokiEvent, EksplorMap) berbagi atribut dan method dari Layanan tanpa duplikasi kode. Polimorfisme

kemudian membuat setiap objek dapat menampilkan informasinya sendiri secara berbeda melalui satu pemanggilan method yang sama — `d.getInfoJoki().tampilDetail()` — yang merupakan inti kekuatan OOP.

Fitur kode promo `DISKONSAHUR` pada `BuildCharacter` mendemonstrasikan constructor overloading secara praktis dan kontekstual — dua constructor dengan parameter berbeda menghasilkan perilaku yang berbeda sesuai kebutuhan.

BAB VI PENUTUP

6.1 Kesimpulan

- Program System Management Joki WUWA berhasil dibangun sebagai aplikasi CRUD berbasis terminal menggunakan Java dengan menerapkan seluruh konsep OOP yang dipersyaratkan.
- Perulangan do-while digunakan untuk menu utama interaktif, sedangkan for (enhanced dan indexed) digunakan untuk iterasi data pesanan sesuai kebutuhan masing-masing operasi.
- Percabangan if-else bertingkat pada class Harga secara efektif mengelola 9 kombinasi harga dari 3 tipe x 3 tingkat layanan. Switch-case pada Main.java menjaga alur menu tetap bersih dan mudah dibaca.
- Enkapsulasi pada DataJoki memastikan integritas data melalui validasi setter. Inheritance memungkinkan code reuse antar class layanan. Polimorfisme method overriding dan constructor overloading membuat program fleksibel dan mudah dikembangkan.

6.2 Saran

Untuk pengembangan lebih lanjut, program ini dapat ditingkatkan dengan menambahkan penyimpanan data persisten menggunakan file atau database sehingga data tidak hilang saat program ditutup. Penambahan fitur pencarian dan filter pesanan, antarmuka grafis (GUI) menggunakan JavaFX, serta sistem notifikasi WhatsApp otomatis juga dapat menjadi pengembangan yang menarik untuk membuat sistem ini lebih siap pakai secara profesional.